

Claude Agents SDK BEATS all Agent Framework! (Beginners Guide)



Mervin Praisson
74.6K subscribers

Join

Subscribe

187



Share

Ask

Download



7,069 views Oct 3, 2025 #ClaudeAI #AIAgents #ClaudeCode

Claude Agents SDK: Build AI Agents with Claude Code Integration | Complete Tutorial

Learn how to harness the power of Claude Agents SDK to create sophisticated AI agents that can solve complex tasks autonomously. In this comprehensive tutorial, I'll show you how to integrate Claude Code—one of the most advanced CLI tools available—into your own Python applications.

<https://mer.vin/2025/10/claude-agent-...>
<https://docs.claude.com/en/docs/claude-...>
<https://docs.claude.com/en/api/agent-...>

What You'll Learn:

- ✓ Install and set up Claude Agents SDK
- ✓ Create a basic AI agent from scratch
- ✓ Integrate internal tools (read/write permissions)
- ✓ Build custom tools and MCP servers
- ✓ Configure Claude Code options for advanced functionality
- ✓ Create a Python web server using AI agents

Timestamps:

- 0:00 - Introduction to Claude Agents SDK
- 0:44 - Installation & Setup
- 1:25 - Creating a Basic Agent
- 2:36 - Using Internal Tools (Read/Write)
- 4:01 - Creating Custom Tools with MCP Servers
- 5:35 - Advanced Claude Options Configuration
- 6:31 - Why Claude Agents SDK Stands Out



Research

Economic Futures

Commitments

Learn

News

Try Claude



Claude Docs

English

Search...

K

Console

Support

Discord

Sign up



Welcome

Claude Developer Platform

Claude Code

Model Context Protocol (MCP)

API Reference

Resources

Release Notes

OpenAI SDK compatibility

Agent SDK

Migrate to Claude Agent SDK

Overview

TypeScript SDK

Python SDK

Guides

Examples

Messages examples

Message Batches examples

AGENT SDK

Agent SDK reference - Python

Copy page

Complete API reference for the Python Agent SDK, including all functions, types, and classes.

Installation

```
pip install claude-agent-sdk
```

Choosing Between `query()` and `Claude`

The Python SDK provides two ways to interact with Claude C

Getting started

Overview

Quickstart

Common workflows

Build with Claude Code

Subagents

Output styles

Hooks

Headless mode

GitHub Actions

GitLab CI/CD

Model Context Protocol (MCP)

GETTING STARTED

Claude Code overview

Copy page ▾

Learn about Claude Code, Anthropic's agentic coding tool that lives in your terminal and helps you turn ideas into code faster than ever before.

Get started in 30 seconds

Prerequisites:

- [Node.js 18 or newer](#)
- A [Claude.ai](#) (recommended) or [Claude Console](#) account

```
# Install Claude Code
npm install -g @anthropic-ai/claude-code

# Navigate to your project
```



What Claude Code does for you

- **Build features from descriptions:** Tell Claude what you want to build in plain English. It will make a plan, write the code, and ensure it works.
- **Debug and fix issues:** Describe a bug or paste an error message. Claude Code will analyze your codebase, identify the problem, and implement a fix.
- **Navigate any codebase:** Ask anything about your team's codebase, and get a thoughtful answer back. Claude Code maintains awareness of your entire project structure, can find up-to-date information from the web, and with [MCP](#) can pull from data sources like Google Drive, Figma, and Slack.
- **Automate tedious tasks:** Fix fiddly lint issues, resolve merge conflicts, and release notes. Do all this in a single command from your developer terminal or automatically in CI.



Getting started

Overview

Quickstart

Common workflows

Build with Claude Code

Subagents

Output styles

Hooks

Headless mode

GitHub Actions

GitLab CI/CD

What Claude Code does for you

- **Build features from descriptions:** Tell Claude what you want to build in plain English. It will make a plan, write the code, and ensure it works.
- **Debug and fix issues:** Describe a bug or paste an error message. Claude Code will analyze your codebase, identify the problem, and implement a fix.
- **Navigate any codebase:** Ask anything about your team's codebase, and get a thoughtful answer back. Claude Code maintains awareness of your entire project structure, can find up-to-date information from the web, and with [MCP](#) can pull from data sources like Google Drive, Figma, and Slack.
- **Automate tedious tasks:** Fix fiddly lint issues, resolve merge conflicts, and release notes. Do all this in a single command from your developer terminal or automatically in CI.



Client SDKs

OpenAI SDK compatibility

Agent SDK ▾

Migrate to Claude Agent SDK

Overview

TypeScript SDK

Python SDK

Guides >

Examples

Messages examples

Message Batches examples

AGENT SDK

Agent SDK reference - Python

Copy page ▾

Complete API reference for the Python Agent SDK, including all functions, types, and classes.

Installation

```
pip install claude-agent-sdk
```

Choosing Between `query()` and `run()`

The Python SDK provides two ways to interact with Claude Code:



CLAUDE AGENTS SDK

1

Basic

2

Internal Tools

3

Custom Tools

4

Claude Options



```
> npm i -g @anthropic-ai/claude-code
```

```
added 9 packages, and changed 3 packages in 1s
```

```
11 packages are looking for funding
  run `npm fund` for details
```

```
> █
```

```
> pip install claude-agent-sdk
```

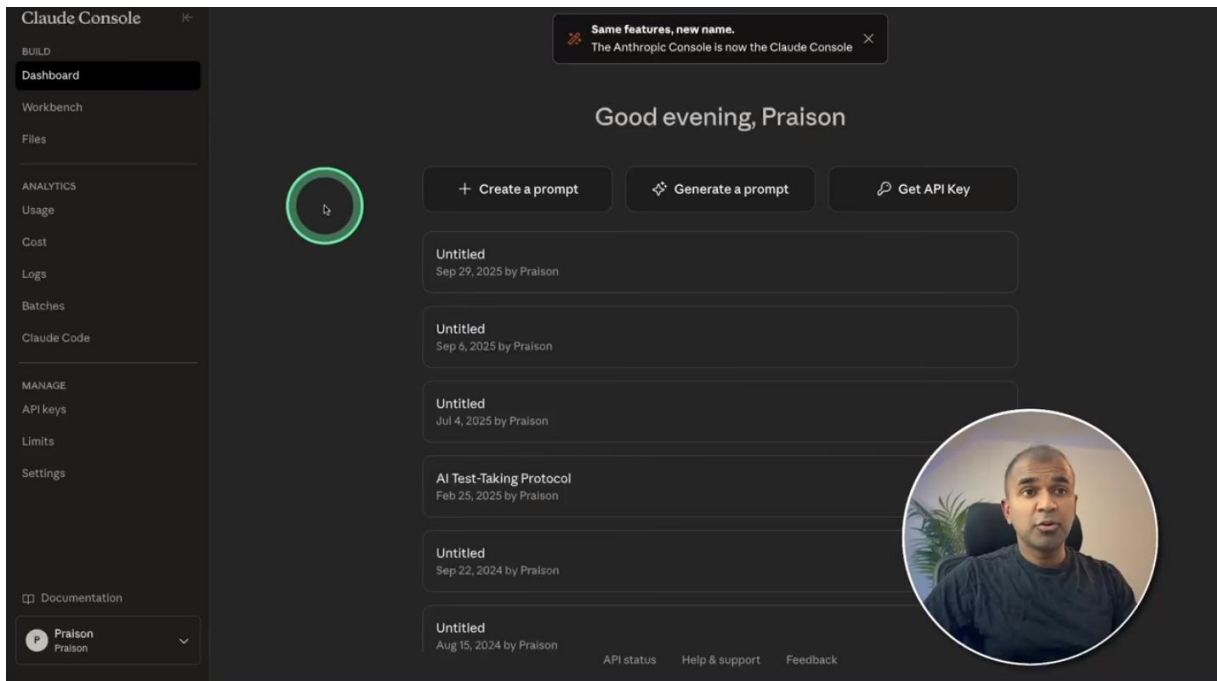
Add to Chat 🗨

```
Requirement already satisfied: claude-agent-sdk in /Users/praison/miniconda3/envs/test/lib/python3.12/site-packages (0.1.0)
```

```
Requirement already satisfied: anyio>=4.0.0 in /Users/praison/miniconda3/envs/test/lib/python3.12/site-packages (from claude-agent-sdk) (4.1.0)
```

```
Requirement already satisfied: mcp>=0.1.0 in /Users/praison/miniconda3/envs/test/lib/python3.12/site-packages (from claude-agent-sdk) (1.16.0)
```

```
> export ANTHROPIC_API_KEY=xxxxxxxxxxxxxxxx█
```



Get this key from platform.openai.com

```
> touch app.py
```



```
app.py
1 import asyncio
2 from claude_agent_sdk import query
3
4 async def main():
5     async for message in query(prompt="Hello, how are you?"):
6         print(message)
7
8 asyncio.run(main())
```



```

> python app.py
SystemMessage(
  subtype='init',
  data={
    'type': 'system',
    'subtype': 'init',
    'cwd': '/Users/praison/cc',
    'session_id': '8029ba33-052d-4950-8ec5-0c78c5a6387d',
    'tools': [
      'Task',
      'Bash',
      'Glob',
      'Grep',
      'ExitPlanMode',
      'Read',
      'Edit',
      'Write',
      'NotebookEdit',
      'WebFetch',
      'TodoWrite',
      'WebSearch',
      'BashOutput',
      'KillShell',
      'SlashCommand'
    ],
    'mcp_servers': [],

```



```

    'pr-comments',
    'release-notes',
    'todos',
    'review',
    'security-review'
  ],
  'apiKeySource': 'ANTHROPIC_API_KEY',
  'output_style': 'default',
  'agents': [
    'general-purpose',
    'statusline-setup',
    'output-style-setup'
  ],
  'uuid': '96a5dbd6-421d-43cd-acf2-95d76b1beced'
}
)
AssistantMessage(
  content=[
    TextBlock(
      text="Hello! I'm doing well, thank you. How can I help you today?"
    )
  ],
  model='claude-sonnet-4-5-20250929',
  parent_tool_use_id=None
)
ResultMessage(

```



```

    'statusline-setup',
    'output-style-setup'
  ],
  'uuid': '96a5dbd6-421d-43cd-acf2-95d76b1beced'
}
)
AssistantMessage(
  content=[
    TextBlock(
      text="Hello! I'm doing well, thank you. How can I help you today?"
    )
  ],
  model='claude-sonnet-4-5-20250929',
  parent_tool_use_id=None
)
ResultMessage(
  subtype='success',
  duration_ms=2748,
  duration_api_ms=2616,
  is_error=False,
  num_turns=1,
  session_id='8029ba33-052d-4950-8ec5-0c78c5a6387d',
  total_cost_usd=0.036115049999999996,
  usage={
    'input_tokens': 3,
    'cache_creation_input_tokens': 9125,

```



```

model='claude-sonnet-4-5-20250929',
parent_tool_use_id=None
)
ResultMessage(
  subtype='success',
  duration_ms=2748,
  duration_api_ms=2616,
  is_error=False,
  num_turns=1,
  session_id='8029ba33-052d-4950-8ec5-0c78c5a6387d',
  total_cost_usd=0.036115049999999996,
  usage={
    'input_tokens': 3,
    'cache_creation_input_tokens': 9125,
    'cache_read_input_tokens': 5291,
    'output_tokens': 20,
    'server_tool_use': {'web_search_requests': 0},
    'service_tier': 'standard',
    'cache_creation': {
      'ephemeral_1h_input_tokens': 0,
      'ephemeral_5m_input_tokens': 9125
    }
  },
  result="Hello! I'm doing well, thank you. How can I help you today?"
)
>

```



2

Internal Tools



```

existing_tools.py
1 import asyncio
2 from claude_agent_sdk import query, ClaudeAgentOptions
3
4 async def main():
5
6     options = ClaudeAgentOptions(
7         allowed_tools=["Read", "Write"],
8         permission_mode="acceptEdits"
9     )
10
11     async for msg in query(
12         prompt="Create a file called greeting.txt with 'Hello Mervin Praison!'",
13         options=options
14     ):
15         print(msg)
16
17 asyncio.run(main())

```



```
> python existing_tools.py
```

```

> python existing_tools.py
SystemMessage(
  subtype='init',
  data={
    'type': 'system',
    'subtype': 'init',
    'cwd': '/Users/praison/cc',
    'session_id': '78c59d5d-3128-4517-abe0-b5138d9087ce',
    'tools': [
      'Task',
      'Bash',
      'Glob',
      'Grep',
      'ExitPlanMode',
      'Read',
      'Edit',
      'Write',
      'NotebookEdit',
      'WebFetch',
      'TodoWrite',
      'WebSearch',
      'BashOutput',
      'KillShell',
      'SlashCommand'
    ],
    'mcp_servers': [],
    'model': 'claude-sonnet-4-5-20250929',

```



```

        'Glob',
        'Grep',
        'ExitPlanMode',
        'Read',
        'Edit',
        'Write',
        'NotebookEdit',
        'WebFetch',
        'TodoWrite',
        'WebSearch',
        'BashOutput',
        'KillShell',
        'SlashCommand'
    ],
    'mcp_servers': [],
    'model': 'claude-sonnet-4-5-20250929',
    'permissionMode': 'acceptEdits',
    'slash_commands': [
        'compact',
        'context',
        'cost',
        'init',
        'output-style:new',
        'pr-comments',
        'release-notes',
        'todos',
        'review',

```



```

    ],
    'mcp_servers': [],
    'model': 'claude-sonnet-4-5-20250929',
    'permissionMode': 'acceptEdits',
    'slash_commands': [
        'compact',
        'context',
        'cost',
        'init',
        'output-style:new',
        'pr-comments',
        'release-notes',
        'todos',
        'review',
        'security-review'
    ],
    'apiKeySource': 'ANTHROPIC_API_KEY',
    'output_style': 'default',
    'agents': ['general-purpose', 'statusline-setup', 'output-style-setup'],
    'uuid': 'd86a08f7-604a-418a-a261-546d482caf7d'
}
)
AssistantMessage(
    content=[
        ToolUseBlock(
            id='toolu_01NafKxyxP6ZphjFJKtB7xaw',
            name='Write',

```



```

        'todos',
        'review',
        'security-review'
    ],
    'apiKeySource': 'ANTHROPIC_API_KEY',
    'output_style': 'default',
    'agents': ['general-purpose', 'statusline-setup', 'output-style-setup'],
    'uuid': 'd86a08f7-604a-418a-a261-546d482caf7d'
}
)
AssistantMessage(
    content=[
        ToolUseBlock(
            id='toolu_01NafKxyxP6ZphjFJKtB7xaw',
            name='Write',
            input={
                'file_path': '/Users/praizon/cc/greeting.txt',
                'content': 'Hello Mervin Praizon!'
            }
        )
    ],
    model='claude-sonnet-4-5-20250929',
    parent_tool_use_id=None
)
UserMessage(
    content=[
        ToolResultBlock(

```



```

)
AssistantMessage(
  content=[
    ToolUseBlock(
      id='toolu_01TXj1DmiVevqFW8z5jvQXqD',
      name='Bash',
      input={
        'command': 'touch greeting.txt && echo 'Hello Mervin Praison!' > greeting.txt',
        'description': 'Create greeting.txt with content'
      }
    )
  ],
  model='claude-sonnet-4-5-20250929',
  parent_tool_use_id=None
)
UserMessage(
  content=[
    ToolResultBlock(
      tool_use_id='toolu_01TXj1DmiVevqFW8z5jvQXqD',
      content='Command contains output redirection (>) which could
files',
      is_error=True
    )
  ],
  parent_tool_use_id=None
)
AssistantMessage(

```



```

  parent_tool_use_id=None
)
AssistantMessage(
  content=[TextBlock(text='\n\nCreated greeting.txt with "Hello Mervin Praison!"')],
  model='claude-sonnet-4-5-20250929',
  parent_tool_use_id=None
)
ResultMessage(
  subtype='success',
  duration_ms=6455,
  duration_api_ms=6283,
  is_error=False,
  num_turns=4,
  session_id='d8fc4765-5c1b-4fad-8229-5668c73b7116',
  total_cost_usd=0.01092555,
  usage={
    'input_tokens': 7,
    'cache_creation_input_tokens': 125,
    'cache_read_input_tokens': 28886,
    'output_tokens': 118,
    'server_tool_use': {'web_search_requests': 0},
    'service_tier': 'standard',
    'cache_creation': {'ephemeral_1h_input_tokens': 0, 'ephemeral_5m_
  },
  result='\n\nCreated greeting.txt with "Hello Mervin Praison!"'
)
>

```



```

greeting.txt
Log: Show Apex Log Analysis
1 Hello Mervin Praison!

```

The file was written successfully using the internally available 'Write' tool



Custom Tools




```
custom_tools.py
1 import asyncio
2 from typing import Any
3 from claude_agent_sdk import ClaudeSDKClient, ClaudeAgentOptions, tool, create_sdk_mcp_se
4
5 async def main():
6     options = ClaudeAgentOptions(
7         mcp_servers={"tools": server},
8         allowed_tools=["mcp_tools_greet"]
9     )
10
11     async with ClaudeSDKClient(options=options) as client:
12         await client.query("Greet Mervin Praisson")
13         async for msg in client.receive_response():
14             print(msg)
15
16 asyncio.run(main())
```

To add a custom tool, you need to set it up as an MCP server

```
custom_tools.py
1
2
3 KClient, ClaudeAgentOptions, tool, create_sdk_mcp_server
4
5
6
7
8 ,
9 greet"]
10
11 is=options) as client:
12 vin Praisson")
13 ve_response():
14
```

```
custom_tools.py
1 import asyncio
2 from typing import Any
3 from claude_agent_sdk import ClaudeSDKClient, ClaudeAgentOptions, tool, create_sdk_mcp_se
4
5 @tool("greet", "Greet a user", {"name": str})
6 async def greet(args: dict[str, Any]) -> dict[str, Any]:
7     return {
8         "content": [{
9             "type": "text",
10             "text": f"Hello, {args['name']}!"
11         }]
12     }
13
14 async def main():
15     options = ClaudeAgentOptions(
16         mcp_servers={"tools": server},
17         allowed_tools=["mcp_tools_greet"]
18     )
19
20     async with ClaudeSDKClient(options=options) as client:
```

We then create a new tool called **greet** as above that we can now integrate with any API

```
custom_tools.py
6 async def greet(args: dict[str, Any]) -> dict[str, Any]:
7     "content": [{
8         "type": "text",
9         "text": f"Hello, {args['name']}!"
10    }]
11 }
12
13
14 server = create_sdk_mcp_server(
15     name="my-tools",
16     version="1.0.0",
17     tools=[greet]
18 )
19
20 async def main():
21     options = ClaudeAgentOptions(
22         mcp_servers={"tools": server},
23         allowed_tools=["mcp_tools_greet"]
24     )
25
26     async with ClaudeSDKClient(options=options) as client:
27         await client.query("Greet Mervin Praison")
```



```
custom_tools.py
14 server = create_sdk_mcp_server(
15     version="1.0.0",
16     tools=[greet]
17 )
18
19
20 async def main():
21     options = ClaudeAgentOptions(
22         mcp_servers={"tools": server},
23         allowed_tools=["mcp_tools_greet"]
24     )
25
26     async with ClaudeSDKClient(options=options) as client:
27         await client.query("Greet Mervin Praison")
28         async for msg in client.receive_response():
29             print(msg)
30
31 asyncio.run(main())
```



```
TERMINAL
python3.12
> python custom_tools.py
```

```
TERMINAL
python3.12
subtype='init',
data={
  'type': 'system',
  'subtype': 'init',
  'cwd': '/Users/praison/cc',
  'session_id': '33e726f0-fd20-4d4f-ab97-a508a0ea40f0',
  'tools': [
    'Task',
    'Bash',
    'Glob',
    'Grep',
    'ExitPlanMode',
    'Read',
    'Edit',
    'Write',
    'NotebookEdit',
    'WebFetch',
    'TodoWrite',
    'WebSearch',
    'BashOutput',
    'KillShell',
    'SlashCommand',
    'mcp_tools_greet'
  ],
  'mcp_servers': [{'name': 'tools', 'status': 'connected'}],
  'model': 'claude-sonnet-4-5-20250929',
  'permissionMode': 'default',
  'slash_commands': [
    'compact',
    'context',
    'cost',
    'init'
  ]
}
```



```
TERMINAL python3.12 + - [ ] ... [ ] x
    'context',
    'cost',
    'init',
    'output-style:new',
    'pr-comments',
    'release-notes',
    'todos',
    'review',
    'security-review'
],
'apiKeySource': 'ANTHROPIC_API_KEY',
'output_style': 'default',
'agents': ['general-purpose', 'statusline-setup', 'output-style-setup'],
'uuid': '819491e0-9d23-4c29-8104-05aba1ff494d'
}
)
AssistantMessage(
  content=[
    ToolUseBlock(
      id='toolu_016NfuVUTuKFNKLeTSmuPQwA',
      name='mcp__tools__greet',
      input={'name': 'Mervin Praisson'}
    )
  ],
  model='claude-sonnet-4-5-20250929',
  parent_tool_use_id=None
)
UserMessage(
  content=[
    ToolResultBlock(
      tool_use_id='toolu_016NfuVUTuKFNKLeTSmuPQwA',
      content=[{'type': 'text', 'text': 'Hello, Mervin Praisson!'}],

```



```
TERMINAL zsh + - [ ] ... [ ] x
    'output_style': 'default',
    'agents': ['general-purpose', 'statusline-setup', 'output-style-setup'],
    'uuid': '819491e0-9d23-4c29-8104-05aba1ff494d'
  }
)
AssistantMessage(
  content=[
    ToolUseBlock(
      id='toolu_016NfuVUTuKFNKLeTSmuPQwA',
      name='mcp__tools__greet',
      input={'name': 'Mervin Praisson'}
    )
  ],
  model='claude-sonnet-4-5-20250929',
  parent_tool_use_id=None
)
UserMessage(
  content=[
    ToolResultBlock(
      tool_use_id='toolu_016NfuVUTuKFNKLeTSmuPQwA',
      content=[{'type': 'text', 'text': 'Hello, Mervin Praisson!'}],
      is_error=None
    )
  ],
  parent_tool_use_id=None
)
AssistantMessage(
  content=[TextBlock(text='\n\nHello, Mervin Praisson!')],
  model='claude-sonnet-4-5-20250929',
  parent_tool_use_id=None
)
ResultMessage(

```



```
TERMINAL
tool_use_id='toolu_016NfuVUTuKFNKLeTSmuPQwA',
content=[{'type': 'text', 'text': 'Hello, Mervin Praison!'}],
is_error=None
)
parent_tool_use_id=None
)
AssistantMessage(
content=[TextBlock(text='\n\nHello, Mervin Praison!')],
model='claude-sonnet-4-5-20250929',
parent_tool_use_id=None
)
ResultMessage(
subtype='success',
duration_ms=6318,
duration_api_ms=7199,
is_error=False,
num_turns=3,
session_id='33e726f0-fd20-4d4f-ab97-a508a0ea40f0',
total_cost_usd=0.042039950000000006,
usage={
'input_tokens': 6,
'cache_creation_input_tokens': 9285,
'cache_read_input_tokens': 19786,
'output_tokens': 75,
'server_tool_use': {'web_search_requests': 0},
'service_tier': 'standard',
'cache_creation': {'ephemeral_1h_input_tokens': 0, 'ephemeral_5m_input_tokens': 0},
},
result='\n\nHello, Mervin Praison!'
)
>
```



```
custom_tools.py
1 import asyncio
2 from typing import Any
3 from claude_agent_sdk import ClaudeSDKClient, ClaudeAgentOptions, tool, create_sdk_mcp_server
4 from rich import print
5
6 @tool("greet", "Greet a user", {"name": str})
7 async def greet(args: dict[str, Any]) -> dict[str, Any]:
8     return {
9         "content": [{
10             "type": "text",
11             "text": f"Hello, {args['name']}!"
12         }]
13     }
14
15 server = create_sdk_mcp_server(
16     name="my-tools",
17     version="1.0.0",
18     tools=[greet]
19 )
20
21 async def main():
22     options = ClaudeAgentOptions(
23         mcp_servers={"tools": server},
24         allowed_tools=["mcp_tools_greet"]
25     )
```



Claude Options




```
create_server.py
1 import asyncio
2 from claude_agent_sdk import query, ClaudeAgentOptions
3
4 async def main():
5     options = ClaudeAgentOptions(
6         system_prompt="You are an expert Python developer",
7         permission_mode='acceptEdits',
8         cwd="/Users/praison/cc"
9     )
10
11     async for message in query(
12         prompt="Create a Python web server",
13         options=options
14     ):
15         print(message)
16
17 asyncio.run(main())
```



English

Search

To exit full screen, press and hold `esc`

Console

Support

Discord

Sign up

Welcome

Claude Developer Platform

Claude Code

Model Context Protocol (MCP)

API Reference

Resources

Release Notes

Client SDKs

OpenAI SDK compatibility

Agent SDK

Migrate to Claude Agent SDK

Overview

TypeScript SDK

Python SDK

Guides

Examples

Messages examples

Message Batches examples

AGENT SDK

Agent SDK reference - Python

Complete API reference for the Python Agent SDK, including all functions, types, and classes.

Copy page

Installation

`pip install claude-agent-sdk`

Choosing Between `query()` and `ClaudeAgentOptions`

The Python SDK provides two ways to interact with Claude C



English

Search

Console

Support

Discord

Sign up

Welcome

Claude Developer Platform

Claude Code

Model Context Protocol (MCP)

API Reference

Resources

Release Notes

Client SDKs

OpenAI SDK compatibility

Agent SDK

Migrate to Claude Agent SDK

Overview

TypeScript SDK

Python SDK

Guides

Examples

Messages examples

Message Batches examples

3rd-party APIs

Any]]]

execution

ClaudeAgentOptions

Configuration dataclass for Claude Code queries.

```
@dataclass
class ClaudeAgentOptions:
    allowed_tools: list[str] = field(default_factory=list)
    system_prompt: str | SystemPromptPreset | None = None
    mcp_servers: dict[str, McpServerConfig] | str | Path | None = None
    permission_mode: PermissionMode | None = None
    continue_conversation: bool = False
    resume: str | None = None
    max_turns: int | None = None
    disallowed_tools: list[str] = field(default_factory=list)
    model: str | None = None
    permission_prompt_tool_name: str | None = None
    cwd: str | Path | None = None
```

```
permission_prompt_tool_name: str | None = None
cwd: str | Path | None = None
settings: str | None = None
add_dirs: list[str | Path] = field(default_factory=list)
env: dict[str, str] = field(default_factory=dict)
extra_args: dict[str, str | None] = field(default_factory=dict)
max_buffer_size: int | None = None
debug_stderr: Any = sys.stderr # Deprecated
stderr: Callable[[str], None] | None = None
can_use_tool: CanUseTool | None = None
hooks: dict[HookEvent, list[HookMatcher]] | None = None
user: str | None = None
include_partial_messages: bool = False
fork_session: bool = False
agents: dict[str, AgentDefinition] | None = None
setting_sources: list[SettingSource] | None = None
```



```
> python create_server.py
```

TERMINAL

python3.12

```
'Edit',
'Write',
'NotebookEdit',
'WebFetch',
'TodoWrite',
'WebSearch',
'BashOutput',
'KillShell',
'SlashCommand'
],
'mcp_servers': [],
'model': 'claude-sonnet-4-5-20250929',
'permissionMode': 'acceptEdits',
'slash_commands': [
    'compact',
    'context',
    'cost',
    'init',
    'output-style:new',
    'pr-comments',
    'release-notes',
    'todos',
    'review',
    'security-review'
],
'apiKeySource': 'ANTHROPIC_API_KEY',
'output_style': 'default',
'agents': ['general-purpose', 'statusline-setup', 'output-style-setup'],
'uuid': '0d6346ae-0618-4ee0-952e-7dfdf3bbff7'
}
```




```
TERMINAL python3.12 + ~ [ ] ... [ ] x
AssistantMessage(
  content=[
    ToolUseBlock(
      id='toolu_01TJrb379a7uYEQMpmeHJbg5',
      name='Write',
      input={
        'file_path': 'server.py',
        'content': '#!/usr/bin/env python3\n"""\nSimple HTTP Web Server\nRun with: python\nserver.py\n"""\n\nfrom http.server import HTTPServer, SimpleHTTPRequestHandler\nimport sys\n\nndef\nrun_server(port=8000):\n    """Start the web server on the specified port."""\n    server_address = ('', port)\n    httpd = HTTPServer(server_address, SimpleHTTPRequestHandler)\n    print(f"Server running on\nhttp://localhost:{port}/")\n    print("Press Ctrl+C to stop the server")\n    try:\n        httpd.serve_forever()\n    except KeyboardInterrupt:\n        print("\nServer stopped.")\n        sys.exit(0)\n\nif __name__ == "__main__":\n    port = 8000\n    if len(sys.argv) > 1:\n        try:\n            port = int(sys.argv[1])\n        except\nValueError:\n            print("Invalid port number. Using default port 8000.")\n        run_server(port)\n'\n      ),
  ],
  model='claude-sonnet-4-5-20250929',
  parent_tool_use_id=None
)
UserMessage(
  content=[
    ToolResultBlock(
      tool_use_id='toolu_01TJrb379a7uYEQMpmeHJbg5',
      content='File created successfully at: server.py',
      is_error=None
    ),
  ],
  parent_tool_use_id=None
)
```



```
TERMINAL python3.12 + ~ [ ] ... [ ] x
default.\n\n**To use a custom port:**\n``bash\npython server.py 3000\n``\n\nThe server will serve files from the\ncurrent directory and can be stopped with `Ctrl+C`. It uses Python's built-in `http.server` module, so no additional\ndependencies are required."
)
],
model='claude-sonnet-4-5-20250929',
parent_tool_use_id=None
)
ResultMessage(
  subtype='success',
  duration_ms=12376,
  duration_api_ms=12193,
  is_error=False,
  num_turns=4,
  session_id='0a157186-87d6-404c-8c75-53f74dbb92e8',
  total_cost_usd=0.018286649999999998,
  usage={
    'input_tokens': 9,
    'cache_creation_input_tokens': 1277,
    'cache_read_input_tokens': 22453,
    'output_tokens': 449,
    'server_tool_use': {'web_search_requests': 0},
    'service_tier': 'standard',
    'cache_creation': {'ephemeral_1h_input_tokens': 0, 'ephemeral_5m_input_tokens': 0}
  },
  result="\n\nI've created a Python web server file called `server.py` in your current directory. To start the server, run the\nserver:**\n``bash\npython server.py\n``\n\nThis will start the server on `http://localhost:8000`. To stop the server, press\n`Ctrl+C`. **\n\nTo use a custom port, run:\n``bash\npython server.py 3000\n``\n\nThe server will serve files from the\ncurrent directory and can be stopped with `Ctrl+C`. It uses Python's built-in `http.server` module, so no additional\ndependencies are required."
)
>
```




```
server.py
1  #!/usr/bin/env python3
2  """
3  Simple HTTP Web Server
4  Run with: python server.py
5  """
6
7  from http.server import HTTPServer, SimpleHTTPRequestHandler
8  import sys
9
10
11 def run_server(port=8000):
12     """Start the web server on the specified port."""
13     server_address = ('', port)
14     httpd = HTTPServer(server_address, SimpleHTTPRequestHandler)
15
16     print(f"Server running on http://localhost:{port}/")
17     print("Press Ctrl+C to stop the server")
18
19     try:
20         httpd.serve_forever()
21     except KeyboardInterrupt:
22         print("\nServer stopped.")
23         sys.exit(0)
24
```



```
server.py
11 def run_server(port=8000):
12     print(f"Server running on http://localhost:{port}/")
13     print("Press Ctrl+C to stop the server")
14
15     try:
16         httpd.serve_forever()
17     except KeyboardInterrupt:
18         print("\nServer stopped.")
19         sys.exit(0)
20
21
22 if __name__ == "__main__":
23     port = 8000
24     if len(sys.argv) > 1:
25         try:
26             port = int(sys.argv[1])
27         except ValueError:
28             print("Invalid port number. Using default port 8000.")
29
30     run_server(port)
31
```



Above is the generated python code that was created for us.

Claude Docs

English

Q Search...

Console Support Discord Sign up

Welcome Claude Developer Platform

Claude Code Model Context Protocol (MCP) API Reference Resources Release Notes

Getting started

Overview Quickstart Common workflows

Build with Claude Code

Subagents Output styles Hooks Headless mode GitHub Actions GitLab CI/CD

GETTING STARTED

Claude Code overview

Learn about Claude Code, Anthropic's agentic coding tool that lives in your terminal and helps you turn ideas into code faster than ever before.

Get started in 30 seconds

Prerequisites:

- Node.js 18 or newer
- A Claude.ai (recommended) or Claude Console account

```
# Install Claude Code
npm install -g @anthropic-ai/claude-code
```